Firmware C trong hệ thống UART

Nios V/m • Avalon-MM → APB • FPGA DE2-115 • bản có FIFO

- Mục tiêu: giải thích từ một lệnh C đến một byte trên chân UART.
- Ứng dụng: chat hai chiều giữa JTAG terminal và picocom; Enter để chuyển dòng.
- Trọng tâm: phân chia phần mềm/phần cứng, truy cập thanh ghi và luồng dữ liệu.
- Tài liệu bám theo source của project; các kết quả kiểm thử có giới hạn được nêu rõ.

GỢI Ý THUYẾT TRÌNH

Giới thiệu bài toán trước, sau đó đặt câu hỏi: CPU gửi một byte ra UART bằng cách nào? Bài trình bày đi từ kiến trúc, qua ví dụ MMIO, đến thuật toán chat và bằng chứng kiểm thử. Các ghi chú ở cuối mỗi trang là gợi ý lời nói, không cần đọc nguyên văn.

Nguồn: firmware/main.c; firmware/dti_uart.c; rtl/uart_avalon.v

Hai CPU, hai vai trò

Chương trình C của project thực thi trên Nios V trong FPGA

- CPU PC chạy VS Code, juart-terminal, picocom và các công cụ build/nạp.
- Nios V/m là CPU mềm trong FPGA: lấy lệnh RISC-V từ RAM và thực thi firmware.
- RAM on-chip 128 KiB chứa mã chương trình, biến, stack và các bộ đệm C.
- TX/RX, FIFO và bus là phần cứng chạy song song với CPU.

GỢI Ý THUYẾT TRÌNH

Không nói chương trình C chạy trên PC để điều khiển trực tiếp từng bit UART. PC biên dịch và nạp chương trình; sau khi chạy, Nios V xử lý các byte, còn TX/RX thực hiện thời gian của frame bằng mạch logic.

Nguồn: fpga/scripts/create_system.tcl; firmware/build.sh; firmware/main.c

Hai terminal đi theo hai đường riêng

Firmware làm cầu chuyển dữ liệu giữa hai ngoại vi

PC: juart-terminal → **USB-Blaster / JTAG** → **IP JTAG UART**

IP JTAG UART → **Nios V + firmware C** → **UART tự thiết kế**

UART TX/RX FPGA → **Chip USB-UART ESP32** → **PC: picocom**

GỢI Ý THUYẾT TRÌNH

JTAG UART là IP trao đổi byte qua JTAG. UART vật lý là khối RTL tự thiết kế, có start/data/stop và baud. Board ESP32 được dùng như mạch USB-UART; CPU ESP32 không chạy chương trình chuyển tiếp của đề tài. Các mũi tên ở đây biểu diễn luồng logic hai chiều.

Nguồn: firmware/terminal_jtag.sh; firmware/terminal_uart.sh; fpga/de2_115_uart_top.v

Firmware điều khiển những gì?

Thao tác trên ngoại vi thông qua thanh ghi ánh xạ bộ nhớ

Đích điều khiển	Thao tác C	Kết quả
UART riêng	CFG, TX, START, RX, STATUS	Cấu hình, gửi/nhận byte
JTAG UART	Đọc DATA; ghi DATA khi WSPACE > 0	Trao đổi với terminal 1
PIO control	Ghi bit loopback / CTS / LED	Chọn đường UART và báo trạng thái
SYSID / timer	Đọc định danh / thời gian	Kiểm tra hệ thống; phát hiện timeout

GỢI Ý THUYẾT TRÌNH

Firmware quyết định dữ liệu, định dạng frame, thời điểm enqueue và cách xử lý lỗi. Nó không tự tạo waveform bằng delay và đổi GPIO. Với bản RTL này, baud được chọn bằng parameter khi build FPGA, không có thanh ghi đổi baud từ C.

Nguồn: firmware/main.c; firmware/dti_uart.c; firmware/console.c; fpga/de2_115_uart_top.v

Avalon là phần cứng kết nối CPU và ngoại vi

C truy cập MMIO; CPU và interconnect thực hiện giao dịch bus

- Avalon-MM là giao diện đọc/ghi theo địa chỉ bên trong FPGA.
- Nios V phát yêu cầu qua data manager; interconnect giải mã địa chỉ để chọn đích.
- UART riêng dùng APB, nên cần bridge Avalon-MM → APB.
- JTAG UART đã có giao diện Avalon-MM và không đi qua bridge APB của UART riêng.

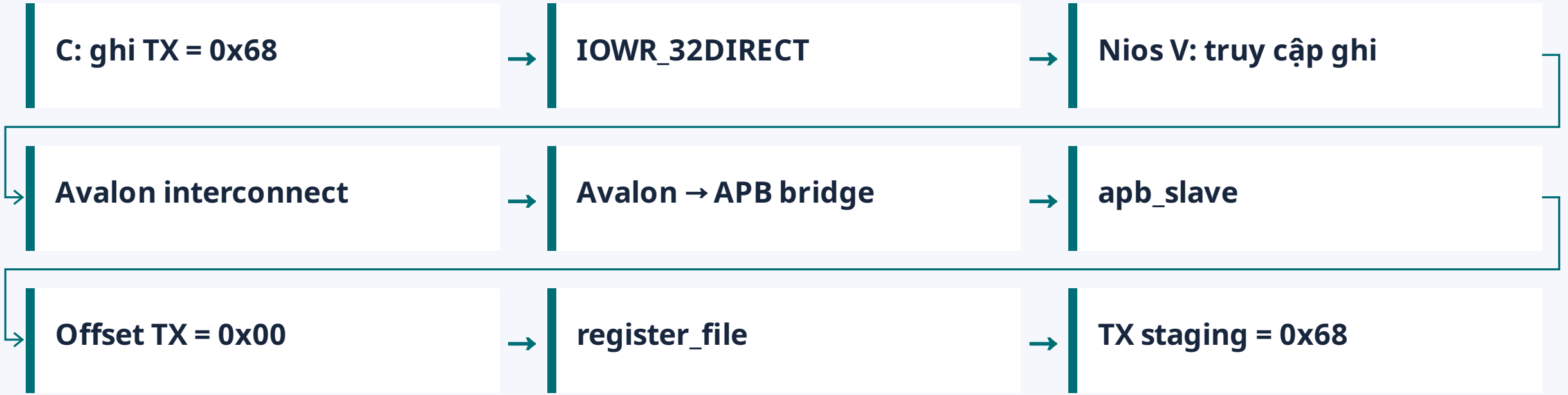
GỢI Ý THUYẾT TRÌNH

Cách diễn đạt chính xác: firmware điều khiển ngoại vi qua MMIO trên hệ thống Avalon-MM. Không nói C là bộ điều khiển giao thức Avalon hay C tự bật PSEL/PENABLE. Tín hiệu bus được phần cứng sinh từ giao dịch đọc/ghi của CPU.

Nguồn: fpga/scripts/create_system.tcl; rtl/uart_avalon.v; rtl/avalon_apb_bridge.v

Một lệnh C đi qua các tầng nào?

Ví dụ: ghi 0x68 vào TX staging register



GỢI Ý THUYẾT TRÌNH

0x68 là mã ASCII của h. Hàm MMIO dùng macro IOWR_32DIRECT của BSP. CPU thực hiện truy cập ghi; interconnect định tuyến đến UART. Địa chỉ toàn hệ thống 0x21000 được nhìn ở UART dưới dạng offset 0x00. Ghi TX chưa enqueue; cần giao dịch START kế tiếp.

Nguồn: firmware/uart_platform.h; firmware/dti_uart.c; rtl/register_file.v

Bản đồ địa chỉ CPU sử dụng

Mỗi vùng địa chỉ chọn một ngoại vi hoặc vùng RAM

Địa chỉ nền	Thành phần	Firmware sử dụng
0x00000000	RAM on-chip 128 KiB	Mã, stack, biến, bộ đệm
0x00020000	JTAG UART	Console hai chiều
0x00020008	System ID	Kiểm tra 0x55415254
0x00020010	PIO control	Loopback, CTS, LED
0x00021000	UART riêng	Base + offset thanh ghi
0x00030000	Timer/software interrupt agent	Đọc mtime theo macro BSP

GỢI Ý THUYẾT TRÌNH

Địa chỉ nền lấy từ system.h do BSP sinh theo hệ thống Platform Designer. Không tự chọn một địa chỉ bất kỳ trong C. Timer có địa chỉ nền 0x30000, còn hàm đọc mtime dùng ALT_CPU_MTIME_OFFSET từ BSP, không mặc định mtime nằm ngay tại địa chỉ nền.

Nguồn: fpga/scripts/create_system.tcl; firmware/build/bsp/system.h; firmware/uart_platform.h

Thanh ghi UART: giao diện giữa C và RTL

Địa chỉ truy cập = UART_BASE + offset

Offset	Thanh ghi	Ý nghĩa đối với firmware
0x00	TX	Ghi byte staging; chưa tự phát
0x04	CFG	Data bits, stop bit, parity; 8N1 = 0x03
0x08	CTRL	Ghi START = 1 để enqueue staging
0x0C	STATUS	Bit 0 TX_DONE; 1 RX_DONE; 2 PARITY_ERROR
0x10	RX	Đọc/pop byte đầu RX FIFO
0x14	FIFO_STATUS	Full/empty, count, RX overrun

GỢI Ý THUYẾT TRÌNH

Ví dụ STATUS có địa chỉ 0x2100C. Đọc RX có side effect: lấy một byte ra khỏi FIFO. Ghi START có side effect: enqueue staging. Vì vậy đây không phải các ô RAM thông thường. Cần đọc đúng trạng thái trước khi truy cập để tránh bus bị giữ chờ.

Nguồn: firmware/dti_uart.h; rtl/register_file.v

Từ MMIO sang APB: một giao dịch ghi

Bridge hiện tại xử lý một yêu cầu tại một thời điểm

- 1. IDLE: nhận yêu cầu Avalon khi sẵn sàng; chốt address, data, byteenable.
- 2. SETUP: bridge đặt PSEL = 1, PENABLE = 0.
- 3. ACCESS: PSEL = 1, PENABLE = 1; giữ yêu cầu đến khi PREADY = 1.
- 4. RESPONSE: trả phản hồi Avalon; lỗi PSLVERR được chuyển thành lỗi bus.

GỢI Ý THUYẾT TRÌNH

Đây là FSM của bridge. APB slave trong project có thêm một wait cycle cố định và có thể chờ thêm theo trạng thái FIFO. Giao dịch chỉ tạo side effect khi được chấp nhận, tránh enqueue lặp trong các chu kỳ chờ. Khi bridge bận, waitrequest chặn nhận yêu cầu Avalon tiếp theo.

Nguồn: rtl/avalon_apb_bridge.v; rtl/apb_slave.v; rtl/register_file.v

Ví dụ thật: enqueue ký tự h

Ba giao dịch MMIO: đọc trạng thái, ghi dữ liệu, ghi lệnh

- 0x21014: kiểm tra TX_FULL trước khi ghi.
- 0x21000: staging nhận 0x68.
- 0x21008: START = 1 chụp staging vào FIFO.
- DTI_OK ở đây nghĩa là đã enqueue, chưa phải frame đã ra hết trên dây.

TRÍCH CODE C

```
if (dti_fifo_status(dev) & DTI_TX_FULL)
    return DTI_AGAIN;
uart_mmio_write(dev->base + DTI_TX, byte);
uart_mmio_write(dev->base + DTI_CTRL, 1u);
return DTI_OK;

// Khi byte = 0x68: enqueue ky tu h
```

GỢI Ý THUYẾT TRÌNH

Đoạn bên phải là thân hàm dti_try_queue trong source. Điều kiện an toàn là một task sở hữu UART: không có ISR hoặc task khác chen vào giữa bước kiểm tra, ghi TX và ghi START. Phần cứng TX tự lấy byte trong FIFO và phát sau đó.

Nguồn: firmware/dti_uart.c: dti_try_queue(); firmware/dti_uart.h

Đã enqueue khác với đã truyền xong

Phần cứng tiếp tục làm việc sau khi lệnh ghi đã hoàn tất

- TX FIFO giữ 16 byte chờ; ngoài ra có 1 byte active đang phát hoặc chờ CTS.
- FIFO_STATUS.TX_EMPTY chỉ báo hàng đợi rỗng; byte active có thể vẫn đang phát.
- STATUS.TX_DONE = 1 khi FIFO rỗng và bộ phát đã hoàn tất.
- Chat chờ TX_DONE cho cả đợt trước khi nạp đợt tiếp theo.

GỢI Ý THUYẾT TRÌNH

Ví dụ FIFO chỉ có h: bộ phát lấy h ra, FIFO đã rỗng nhưng TX vẫn đang gửi start/data/stop. Nếu dùng TX_EMPTY để kết luận truyền xong thì có thể kết luận quá sớm. TX_DONE là điều kiện driver dùng khi cần đợi toàn bộ phát hết.

Nguồn: rtl/register_file.v: tx_done; firmware/main.c: uart_demo_poll()

RX: đọc đúng thứ tự để giữ đúng parity

STATUS mô tả byte đầu FIFO; đọc RX sẽ chuyển sang byte kế tiếp

- Kiểm tra RX_DONE để tránh đọc FIFO rỗng và giữ bus chờ.
- Lưu STATUS trước khi pop RX.
- Trả byte cùng kết quả parity của chính byte vừa đọc.
- RX_DONE vẫn có thể bằng 1 sau lần đọc nếu FIFO còn byte.

TRÍCH CODE C

```
uint32_t status = dti_status(dev);
if (!(status & DTI_RX_DONE))
    return DTI_AGAIN;
*byte = (uint8_t)uart_mmio_read(
    dev->base + DTI_RX);
return (status & DTI_PARITY_ERROR)
    ? DTI_PARITY : DTI_OK;
```

GỢI Ý THUYẾT TRÌNH

Không đọc RX trước rồi mới đọc parity: khi đó STATUS có thể đã mô tả byte kế tiếp. Byte có lỗi parity vẫn được driver trả về và lấy khỏi FIFO; ứng dụng chat chọn bỏ dòng đang nhập. Frame sai stop bị RX core loại ở phần cứng.

Nguồn: firmware/dti_uart.c: dti_try_receive(); rtl/register_file.v

Các lớp mã C và trách nhiệm

Tách chính sách ứng dụng khỏi truy cập thanh ghi

File	Trách nhiệm	Ví dụ
main.c	Luồng chat, dòng nhập, queue, lỗi	uart_demo_init / uart_demo_poll
dti_uart.c/.h	Driver UART riêng	configure / try_queue / try_receive
console.c/.h	Driver console JTAG và queue	console_try_receive / console_poll
uart_platform.h	MMIO và đọc timer	IORD_32DIRECT / IOWR_32DIRECT
system.h + BSP	Địa chỉ, clock, nền chạy chương trình	UART_BASE / ALT_CPU_FREQ

GỢI Ý THUYẾT TRÌNH

Khi trình bày code, bắt đầu từ main rồi đi xuống driver và MMIO. main quyết định Enter mới gửi; driver biết offset và điều kiện FIFO; uart_platform biết cách thực hiện I/O trên Nios V. system.h cung cấp cấu hình phần cứng do BSP sinh.

Nguồn: firmware/main.c; firmware/dti_uart.c; firmware/console.c; firmware/uart_platform.h

Khởi tạo: đưa hệ thống về trạng thái xác định

uart_demo_init() chạy một lần trước vòng lặp polling

- Xóa trạng thái phần mềm: queue, bộ đệm dòng và cờ lỗi.
- Ghi CONTROL = 0: chọn UART ngoài, không sử dụng CTS ngoài; khởi tạo console.
- Kiểm tra SYSID; dti_init lưu base/timeout; dti_configure chọn 8N1.
- Nếu thành công: bật LED ready và đưa thông báo READY vào console JTAG.

GỢI Ý THUYẾT TRÌNH

dti_init không reset hay cấu hình phần cứng: nó kiểm tra tham số và lưu vào struct. dti_configure mới đợi TX_DONE và ghi CFG. SYSID là một kiểm tra định danh cơ bản; giá trị cố định này không chứng minh mọi phiên bản RTL/firmware đều tương thích.

Nguồn: firmware/main.c: uart_demo_init(); firmware/dti_uart.c; fpga/de2_115_uart_top.v

Cấu hình 8N1: từ tham số C đến CFG

dti_configure() kiểm tra tham số, đợi TX xong rồi ghi thanh ghi

- bits = 8 → trường data bits bằng 8 − 5 = 3.
- stops = 1 → trường stop bằng 1 − 1 = 0.
- Không parity → parity enable = 0; tổng CFG = 0x03.
- Baud 115200 là parameter RTL; hàm này không đổi baud.

TRÍCH CODE C

```
// Loi goi trong uart_demo_init():
dti_configure(&uart, 8, 1,
              DTI_PARITY_NONE);

// Cong thuc trong driver:
uint32_t cfg = (bits - 5u)
             | ((stops - 1u) << 2);
// 8N1: cfg = 3 = 0x03
```

GỢI Ý THUYẾT TRÌNH

CFG bit [1:0] mã hóa 5–8 data bit, bit 2 chọn 1 hoặc 2 stop, bit 3 bật parity, bit 4 chọn even khi bằng 1. Driver đợi TX_DONE để giữ nguyên format của các byte đã queue. Phía gửi RX cũng phải idle khi đổi format vì driver không có bit trạng thái RX busy.

Nguồn: firmware/dti_uart.c: dti_configure(); rtl/register_file.v

Vòng lặp chính: polling có giới hạn công việc

Mỗi lượt phục vụ cả JTAG, RX và TX

- console_poll(): xuất tối đa 16 byte JTAG khi IP còn chỗ.
- Kiểm tra lỗi fatal, hoàn tất TX hoặc timeout; kiểm tra RX overrun.
- Thử lấy tối đa 1 byte UART RX và 1 byte JTAG để xử lý dòng.
- Nếu TX đợt trước đã xong: enqueue tối đa 16 byte đang chờ; lặp lại.

GỢI Ý THUYẾT TRÌNH

Ứng dụng không dùng RTOS hay ngắt để phục vụ dữ liệu. Dây IRQ của JTAG UART có trong hệ thống nhưng console_init tắt interrupt IP. Từ “không chặn” ở đây nghĩa là tránh chờ frame dài hoặc chờ dữ liệu chưa có; mỗi MMIO vẫn cần các chu kỳ bus để hoàn tất.

Nguồn: firmware/main.c: main(), uart_demo_poll(); firmware/console.c

Điều kiện Enter nằm trong firmware

Hai input_line độc lập, mỗi dòng tối đa 120 byte

- Ký tự thường: echo về bên đang nhập và thêm vào bộ đệm dòng.
- CR hoặc LF: kết thúc dòng, thêm CRLF và chuyển sang hàng đợi bên kia.
- CR rồi LF: skip_lf bỏ LF đi kèm, tránh tạo thêm một dòng trống.
- Backspace/DEL: sửa byte cuối khi dòng chưa bị đánh dấu loại bỏ.

GỢI Ý THUYẾT TRÌNH

Enter không làm UART phần cứng chuyển chế độ. Picocom vẫn gửi từng phím qua dây, CPU lưu dòng cho đến Enter. CR đưa con trỏ về đầu dòng; LF xuống dòng. Chuẩn hóa thành CRLF giải quyết hiện tượng chữ lệch dần; bỏ LF sau CR tránh xuống hai lần.

Nguồn: firmware/main.c: input_line, receive_char(); firmware/terminal_uart.sh

Ví dụ end-to-end: JTAG → picocom

Người dùng gõ hello rồi Enter ở terminal 1

JTAG UART: byte đến → C: jtag_line = hello → Enter → hello + CRLF

Queue C: 512 byte → dti_try_queue() → Avalon → APB → TX FIFO

TX core: phát frame → USB-UART trên ESP32 → Picocom: hiển thị dòng

GỢI Ý THUYẾT TRÌNH

Trước Enter, chữ chỉ được echo về terminal đang nhập. Sau Enter, chuỗi 7 byte 68 65 6C 6C 6F 0D 0A đi vào queue UART rồi TX FIFO. CPU ghi thanh ghi qua Avalon/APB; waveform được TX core tạo. Không có đường nối tắt JTAG UART sang TX FIFO bỏ qua firmware.

Nguồn: firmware/main.c: receive_char(), queue_write(), uart_demo_poll(); firmware/console.c

Ví dụ end-to-end: picocom → JTAG

Nhận UART và echo tại chỗ cũng cần firmware xử lý



GỢI Ý THUYẾT TRÌNH

Picocom tắt local echo, vì vậy ký tự hiện lại ở terminal 2 là phản hồi đi qua FPGA. Firmware nhận byte từ RX FIFO, echo qua queue TX, đồng thời giữ vào uart_line. Đến Enter, dòng hoàn chỉnh được đưa vào console_write để chuyển ra JTAG.

Nguồn: firmware/main.c: local_echo(), receive_char(); firmware/console.c

JTAG console cũng là ngoại vi MMIO

Giao diện phía CPU là Avalon-MM; phía PC là JTAG

- Đọc DATA tại base + 0: RVALID bit 15 cho biết byte có hợp lệ không.
- Byte nhận nằm ở DATA[7:0]; console_try_receive kiểm tra trước khi trả về.
- Đọc CONTROL tại base + 4: WSPACE ở [31:16] cho biết chỗ trống phía phát.
- console_poll chỉ ghi DATA khi WSPACE > 0; tối đa 16 byte mỗi lượt.

GỢI Ý THUYẾT TRÌNH

Queue console 512 byte là mảng trong RAM, khác bộ đệm bên trong IP JTAG UART. Khi terminal JTAG không tiêu thụ dữ liệu, firmware giữ phần chờ trong queue và vẫn quay lại phục vụ UART. Tuy nhiên queue hữu hạn nên cuối cùng vẫn có thể đầy; không cam kết lưu vô hạn.

Nguồn: firmware/console.c: console_try_receive(), console_poll(); fpga/scripts/create_system.tcl

Các bộ đệm giải quyết những vấn đề khác nhau

Phân biệt phần mềm trong RAM và FIFO phần cứng

Lớp đệm	Dung lượng	Mục đích
Dòng JTAG / dòng UART trong RAM	120 byte mỗi dòng	Chờ Enter, xử lý sửa dòng
Queue TX UART trong RAM	512 byte	Chờ nạp xuống FIFO
Queue console JTAG trong RAM	512 byte	Chờ IP JTAG còn chỗ
TX FIFO phần cứng	16 byte + 1 active ngoài FIFO	Nuôi bộ phát tự động
RX FIFO phần cứng	16 byte + parity từng byte	Giữ dữ liệu chờ CPU đọc

GỢI Ý THUYẾT TRÌNH

Bộ đệm dòng quyết định chính sách giao tiếp người dùng. Queue C hấp thụ chênh lệch tốc độ xử lý và tốc độ đầu ra. FIFO phần cứng cho phép core UART chạy trong khi CPU đang làm việc khác. Phát 16 byte 8N1 mất khoảng 1,38 ms với bit 8,64 µs, chưa tính khoảng nghỉ.

Nguồn: firmware/main.c; firmware/console.c; rtl/register_file.v; rtl/uart_fifo.v

Lỗi và timeout: phản ứng có chủ đích

Firmware phân biệt lỗi có thể tiếp tục và lỗi cần reset

Tình huống	Phản ứng	Phạm vi
Parity / dòng quá 120 byte	Đánh dấu bỏ dòng đến Enter	Có thể nhận dòng tiếp theo
Queue đích không đủ chỗ	Bỏ cả dòng gửi sang đích; bật LED lỗi	Không enqueue một phần dòng
RX FIFO overrun	Báo lỗi fatal; yêu cầu reset	Byte mới đã bị mất
TX chưa xong sau 100 ms	Báo lỗi fatal; yêu cầu reset	Không tự gửi lại gây trùng
Sai SYSID / lỗi khởi tạo	Không chạy chat; báo lỗi	Cần kiểm tra hệ thống

GỢI Ý THUYẾT TRÌNH

Timeout 100 ms là phép đo thời gian phần mềm, không phải bộ giải cứu một bus đã treo. Driver kiểm tra FIFO trước để tránh truy cập blocking trong hoạt động bình thường. Sau lỗi fatal, vòng lặp vẫn gọi console_poll để cố xuất thông báo nhưng dừng xử lý chat; không cưỡng bức hủy frame phần cứng đã chạy.

Nguồn: firmware/main.c: report_error(), uart_demo_poll(); firmware/uart_platform.h

Timeout: đo thời gian mà không delay vòng lặp

Timer mtime cung cấp mốc thời gian cho một đợt TX

- TX_TIMEOUT = ALT_CPU_FREQ / 10: 100 ms tại 50 MHz.
- Enqueue xong: lưu tx_started, quay lại polling.
- TX_DONE: xong; hết thời hạn: lỗi fatal.
- Đọc mtime thấp 32 bit; trừ unsigned để xử lý wrap.

TRÍCH CODE C

```
// Trich dieu kien trong vong poll:
(uint32_t)(uart_cycles() - tx_started)
    >= TX_TIMEOUT

// Nguon thoi gian trong wrapper:
return IORD_32DIRECT(
    ALT_CPU_MTIME_OFFSET, 0);
```

GỢI Ý THUYẾT TRÌNH

Tên uart_cycles là tên wrapper của project; nguồn đọc thực tế là mtime qua MMIO, không phải CSR mcycle. Theo cấu hình này mtime tăng mỗi clock CPU. Phép trừ unsigned dùng được cho timeout ngắn với việc kiểm tra thường xuyên. Nó không giúp thoát nếu chính giao dịch MMIO đã bị phần cứng giữ chờ.

Nguồn: firmware/main.c: TX_TIMEOUT, uart_demo_poll(); firmware/uart_platform.h

Ba API gửi: dùng đúng theo mục đích

Chat theo lô; scope và self-test có thể gửi từng byte

API	Điều kiện bắt đầu	Ý nghĩa khi trả DTI_OK
dti_try_queue()	TX FIFO còn chỗ	Đã enqueue; chưa bảo đảm phát xong
dti_try_send()	TX_DONE = 1	Đã enqueue một byte khi TX idle
dti_send()	Đợi TX trước và sau khi gửi	Byte đã phát xong
dti_wait_tx()	Polling TX_DONE đến timeout	Toàn bộ TX đã phát xong
dti_try_receive()	RX_DONE = 1	Đã pop byte; parity có mã lỗi riêng

GỢI Ý THUYẾT TRÌNH

dti_send gọi chờ TX, gửi rồi chờ hoàn tất. dti_try_send chỉ nhận byte mới khi toàn bộ TX đã idle, nhưng trả về sau khi queue lệnh. dti_try_queue tận dụng FIFO khi vẫn đang bận. Tránh dùng dti_send cho mọi byte trong vòng lặp chat vì thời gian chờ có thể trì hoãn phục vụ RX.

Nguồn: firmware/dti_uart.c; firmware/main.c; firmware/examples/scope_main.c

Nạp phần cứng và chương trình là hai bước

SOF tạo hệ thống; ELF cung cấp chương trình cho Nios V

- RTL + Platform Designer → Quartus → .sof: CPU, RAM, bus, UART và kết nối.
- C + BSP → RISC-V compiler/linker → .elf: lệnh máy và dữ liệu ứng dụng.
- program.sh nạp SOF qua JTAG rồi dùng niosv-download để nạp/chạy ELF.
- Sửa FIFO cần SOF mới; firmware hiện đọc 0x14 nên phải đi cùng RTL hỗ trợ nó.

GỢI Ý THUYẾT TRÌNH

C không tạo ra Avalon bridge khi chạy. Bridge đã tồn tại sau khi cấu hình FPGA. Không cần mở GUI Quartus để chat. Trong quy trình hiện tại, nạp lại khi mất cấu hình/bộ nhớ; bài này không triển khai cơ chế tự boot ứng dụng từ flash.

Nguồn: fpga/program.sh; fpga/build.sh; firmware/build.sh

Bằng chứng kiểm thử và điều chưa chứng minh

Tách kết quả mô phỏng, cross-compile và hoạt động trên kit

- RTL regression: 13/13 PASS; FIFO có 30.000 chu kỳ ngẫu nhiên ở depth 1/3/16.
- Driver C + RTL: 192 frame PASS; ứng dụng chat C + RTL: 528 byte đầu ra PASS.
- ELF echo/selftest/scope build PASS; Quartus constrained slack cuối: +0,105 ns.
- Bản FIFO mới chưa được xác nhận trên kit trong lần kiểm thử được ghi lại.

GỢI Ý THUYẾT TRÌNH

Các bài C+RTL dùng C native và UART RTL, không chạy ELF trên mô hình CPU Nios V. Full CPU simulation bị chặn quyền socket trong môi trường công cụ. Timing PASS không đồng nghĩa license hợp lệ hay mọi đường ngoài chip đã được đo. Chỉ báo cáo hardware PASS sau khi nạp đúng bản và thử thực tế.

Nguồn: firmware/VALIDATION.md; sim/results/; fpga/build/logs/compile.log

Kịch bản demo và các câu hỏi thường gặp

Dùng hành vi quan sát được để giải thích vai trò firmware

- Demo: gõ hello trên JTAG → chỉ hiện tại chỗ; Enter → picocom nhận cả dòng.
- Đổi chiều, thử CR/LF, Backspace và nhiều dòng: chứng minh hai bộ đệm độc lập.
- “C điều khiển Avalon?” → C truy cập MMIO; CPU phát bus, bridge chuyển giao thức.
- “Vì sao cần FIFO?” → giảm yêu cầu CPU phục vụ đúng thời điểm từng byte, vẫn có giới hạn.

GỢI Ý THUYẾT TRÌNH

Có thể kết thúc bằng một câu: firmware quyết định dữ liệu và chính sách xử lý; bus vận chuyển giao dịch; UART phần cứng quyết định dạng sóng. Nếu được hỏi tại sao chưa dùng interrupt/DMA: hiện dùng polling đơn giản, dễ kiểm chứng; nâng cấp chỉ khi yêu cầu tải và độ trễ cần đến.

Nguồn: firmware/main.c; firmware/console.c; rtl/avalon_apb_bridge.v; docs/FIFO.md

Nguồn đối chiếu và cách dùng bộ slide

Tài liệu mô tả source hiện tại, không thay thế spec giao thức đầy đủ

- Kiến trúc: fpga/scripts/create_system.tcl; fpga/de2_115_uart_top.v.
- Firmware: main.c, dti_uart.c/.h, console.c/.h, uart_platform.h trong firmware/.
- Bus và FIFO: rtl/avalon_apb_bridge.v, apb_slave.v, register_file.v, uart_fifo.v.
- Kiểm chứng: firmware/VALIDATION.md; docs/FIFO.md; script/testbench trong sim/ và tb/.

GỢI Ý THUYẾT TRÌNH

Gợi ý: dùng phần kiến trúc, MMIO, ví dụ TX/RX và luồng chat làm bài chính; dùng bảng API, timer và kiểm thử để giải thích sâu hoặc trả lời. Code trong trang 10 và 12 trích thân hàm hiện tại, chỉ đổi cách xuống dòng. Chỉ số và kết luận kiểm thử là dữ liệu đã ghi lại, không phải phép đo mới trong lần tạo PDF.

Nguồn: Đối chiếu source ngày 15/10/2026; mọi đường dẫn tương đối tính từ UART/